CROSS PLATFORM APP DEVELOPMENT (CPAD) · JURY 1

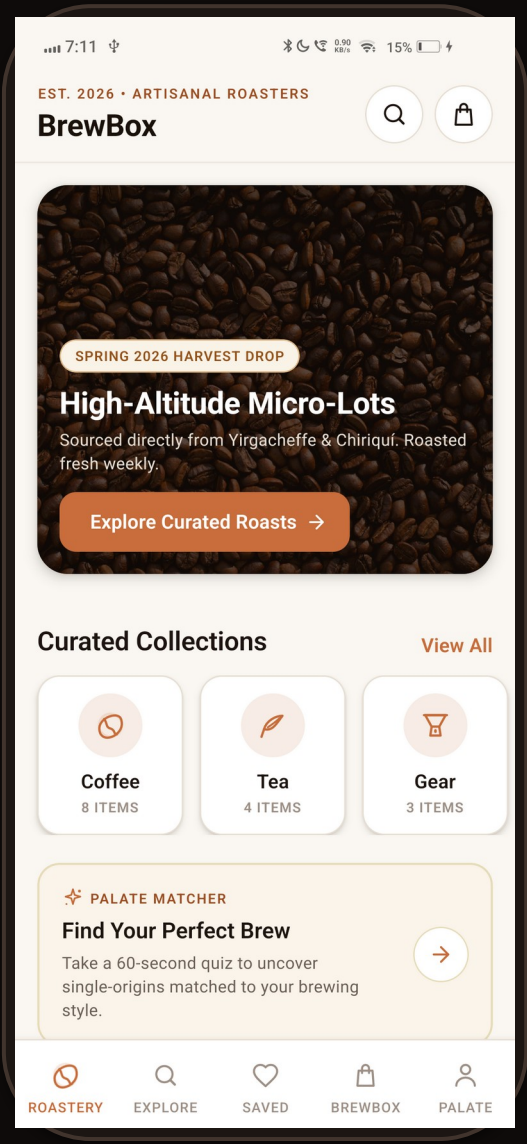
BrewBox

Specialty Coffee & Fine Tea Roastery
Cross-Platform Mobile Application in React Native

Varad Vikas Mandhare

B.Tech Information Technology · MIT ADT University, Pune

React Native 0.87.0 · TypeScript 6.0 · Zustand 5.0 · Hermes Engine · Android SDK



Why React Native for BrewBox?

Evaluating mobile architecture trade-offs for high-performance cross-platform commerce.

01

Native Silos (Kotlin / Swift)

Maintaining two distinct native codebases requires duplicated business logic, divergent cart mathematics, and fragmented feature rollouts.

High engineering overhead for specialty single-origin stores where consistency across Android and iOS is critical.

02

Hybrid WebViews (Cordova/PWA)

Web-based wrappers suffer from DOM bridge latency, stuttering scroll views, lack of 60fps tactile feedback, and fragile offline behavior.

Incapable of delivering the rich, sensory roastery experience (5-stage RoastMeters, extraction timers) required.

03

React Native (BrewBox)

Single declarative TypeScript codebase compiled to real native Android/iOS view hierarchies via the Yoga Flexbox engine.

Hermes AOT bytecode execution, zero DOM overhead, unidirectional Zustand state, and 100% standalone offline APK capability.

- CPAD Core Principle: Shared declarative TypeScript logic paired with native platform UI rendering for 60fps performance.

Six engineering goals in React Native.

01
Cross-Platform Architecture
Single declarative TypeScript codebase running natively on Android with zero code duplication or platform silos.

02
Custom Component Design
Handcrafted artisanal UI widgets without heavy third-party UI kits (RoastMeter, TasteProfileBar, GrindSelector).

03
Declarative State Management
Lightweight unidirectional state using Zustand 5.0, eliminating prop-drilling and optimizing render performance.

04
Multi-Screen Navigation
Deeply nested routing architecture combining React Navigation Native Stack and custom 5-tab Bottom Tab Bar.

05
Vector Graphics Pipeline
Zero-dependency vector rendering using react-native-svg, ensuring scalable high-DPI rendering across devices.

06
Native Build & Offline APK
Complete React Native CLI workflow, embedded JS bundling, and zero-Metro standalone offline execution on hardware.

Modern React Native ecosystem, end to end.

REACT NATIVE CORE

React Native 0.87.0
New Architecture ready

TypeScript 6.0 Strict
100% type safety

Hermes JS Engine
Ahead-Of-Time (AOT) bytecode

Yoga Flexbox Engine
Cross-platform flex layout

STATE & NAVIGATION

Zustand 5.0
Unidirectional reactive stores

React Navigation 7.x
Native Stack & Bottom Tabs

React Native SVG
Scalable vector pipeline

Zero UI Kits
Handcrafted artisanal tokens

BUILD & PLATFORM

Android Studio & SDK
Target API 34 / 35

Vivo 1919 Device
Physical Android 12 testing

Gradle 9 & JDK 21
Native C++ CMake build

Standalone Bundle
Embedded assets (useDevSupport=false)

TESTING & QUALITY

Jest 29 & Renderer
Unit tests for stores & math

ESLint
Zero warnings, clean quality

Git & GitHub
Public repository & CI

Strict Compiler
Zero errors in npx tsc

● Bare React Native Community CLI workflow · Zero Expo bloat · Native Android SDK · Direct access to native Host

Hierarchical React Native component design.

COMPONENT & NAVIGATION TOPOLOGY

```
SafeAreaProvider  (Edge-to-edge Android display)
  └── NavigationContainer  (React Navigation 7 Context)
        └── RootNavigator  (Native Stack Navigator)
              ├── BottomTabNavigator  (Custom 5-Tab Bar)
              │     ├── RoasteryScreen  (Hero, Micro-Lots)
              │     ├── ExploreScreen  (Filters & Search)
              │     ├── SavedScreen  (Wishlist Favorites)
              │     ├── CartScreen  (Items & Shipping Meter)
              │     └── PalateScreen  (Profile & Quiz)
              ├── ProductDetails  (RoastMeter & Dial-In Guide)
              ├── CheckoutScreen  (Batch Scheduling & Pay)
              └── OrderTracking  (5-Stage Roastery Timeline)
```

REACT NATIVE CORE APIS UTILIZED

`<View> & <Text>`
Semantic structural primitives with strict typography hierarchy

`<ScrollView> & <FlatList>`
Virtualized lists with optimized keyExtractors & lazy rendering

`<Pressable>`
Modern touch handling with active tactile opacity & ripple

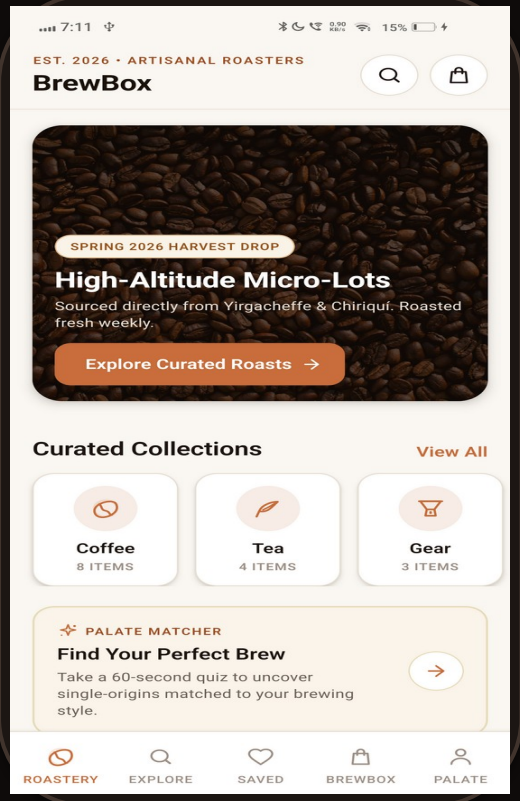
`<Modal>`
Native animated overlay for 3-step Palate Matcher Quiz

`StyleSheet.create()`
Cached, validated style objects compiled to native IDs

`Svg, Path, Circle`
Zero-dependency vector rendering via react-native-svg

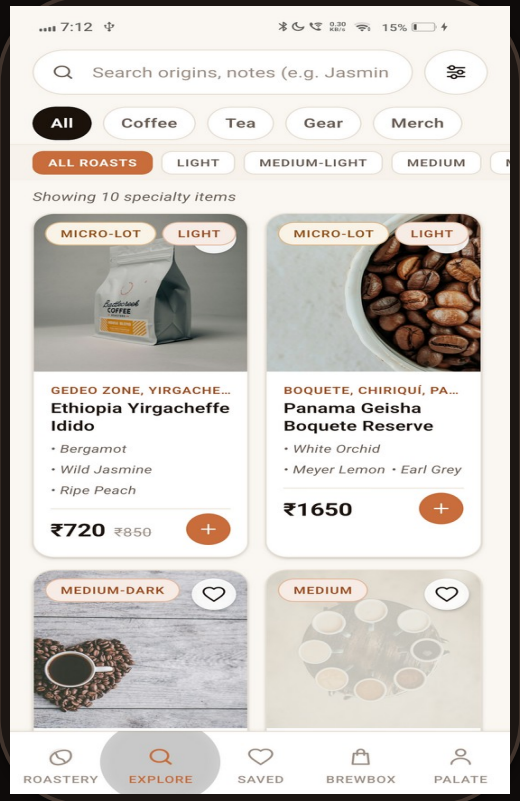
`Platform & Dimensions`
Dynamic device notch, safe insets, and screen scaling

Cross-platform screens in live execution.



01 Home Screen

FlatLists, hero banner & collections



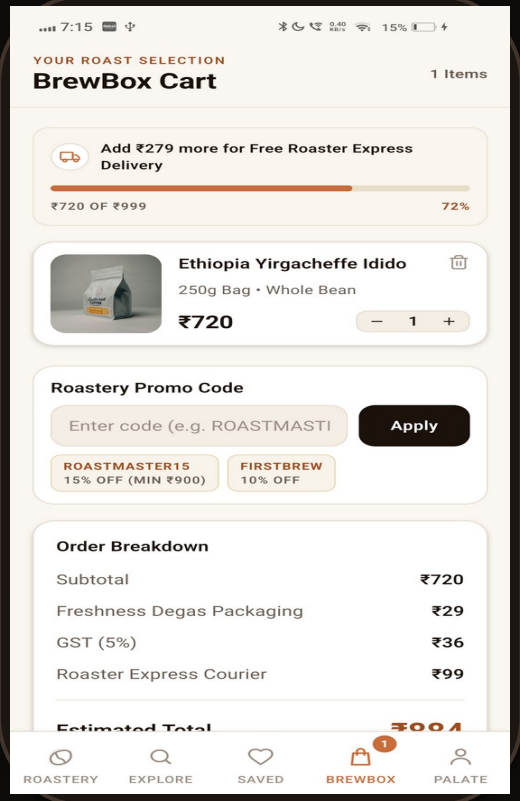
02 Explore Screen

Controlled TextInput & roast filters



03 Details Screen

Custom SVG RoastMeter & dial-in guide



04 Cart Screen

Derived state, free shipping progress

Unidirectional data flow with Zustand in React Native.

STATE MANAGEMENT PARADIGM COMPARISON

- React Context API: Triggers full sub-tree re-renders on any state mutation, causing noticeable frame drops in lists.
- Redux: Heavy boilerplate, actions/reducers ceremony, and oversized bundle footprint.
- **Zustand 5.0 (BrewBox Choice): Micro-store architecture with selective subscriptions (useCartStore(s => s.items)), immutable updates, zero boilerplate, and 60fps UI rendering.**

`cartStore.ts`

Cart & Commerce

- Item variants (Grind, Weight)
- Dynamic quantity steppers
- Free shipping meter (₹999)
- 5% GST tax calculation
- Promo vouchers (ROASTMASTER15)

`wishlistStore.ts`

Favorites & Bookmarks

- O(1) Set lookup by ID
- Optimistic UI toggling
- One-tap 'Move to Cart'
- Local device persistence
- Zero unnecessary re-renders

`filterStore.ts`

Catalog Predicates

- Real-time search text filter
- Category tabs (Coffee, Tea)
- Roast level multi-select
- Micro-lot exclusivity flag
- Dynamic sorting algorithms

`userStore.ts`

Profile & Orders

- Roastery Gold member tier
- Bean Rewards accumulation
- Saved delivery addresses
- Palate Persona taste leaning
- 5-Stage live order batch tracking

Artisanal cross-platform design system.

Leveraging React Native's Yoga Flexbox engine for consistent cross-platform layouts.

REACT NATIVE STYLING PRINCIPLES

01 Yoga Flexbox Layout — Pure Flexbox layout (flexDirection, justifyContent, alignItems, flexWrap) working identically across Android and iOS.

02 Compiled StyleSheet API — Styles defined outside render functions via StyleSheet.create(), compiled into native style IDs to reduce JS-to-native overhead.

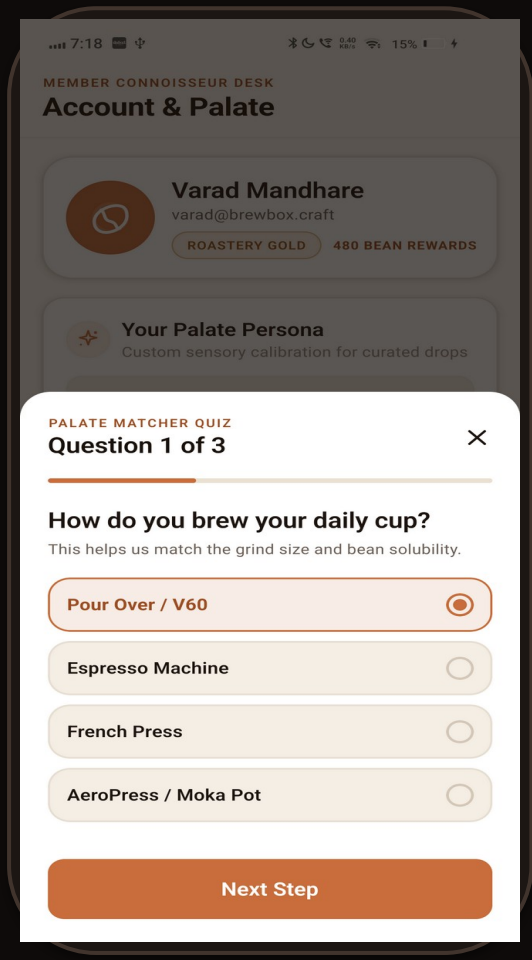
03 Artisanal Design Tokens — Centralized palette tokens in src/theme/ (Terracotta #C86D3B, Obsidian #1A120B, Oat Cream #F9F6F0, Matcha Sage #2E4A35).

04 Bespoke Vector Pipeline — 30+ custom SVG glyphs (bean, dripper, teapot, flame) built with react-native-svg — zero icon font dependency.

05 Interactive Sensory Widgets — Custom visual 5-stage RoastMeter, multi-dimensional TasteProfileBar, and tactile 6-stage GrindSelector.

06 Platform Shadow Adaptation — Cross-platform depth elevation using Android elevation attribute alongside iOS shadowColor/shadowOffset tokens.

07 Responsive SafeArea Design — Seamless handling of Android status bar, camera hole-punch cutouts, and navigation gestures.



09 — PERFORMANCE & RUNTIME OPTIMIZATION

Engineering 60fps cross-platform interactions.

01
Hermes JavaScript Engine
Pre-compiles JavaScript into optimized bytecode Ahead-Of-Time (AOT). Eliminates runtime JS parsing, cuts app startup time (TTI) by ~50%, and minimizes RAM consumption.

02
Atomic Zustand Selectors
Components subscribe only to exact state slices (e.g. useCartStore(s => s.itemCount)), preventing parent re-renders from cascading down the virtual DOM tree.

03
StyleSheet Caching
Styles are declared once outside component lifecycles using StyleSheet.create(), avoiding memory allocations on every state update and caching style IDs on the native side.

04
Pressable API with Ripples
Uses modern React Native Pressable with hardware-accelerated ripple effects on Android and instant gesture response without touch delay.

05
FlatList Virtualization
Employs getItemLayout, initialNumToRender, and unique keyExtractors to recycle off-screen views, maintaining 60fps scrolling in product grids.

06
Zero-Dependency SVGs
Custom vector paths eliminate heavy font icon bundles or remote image requests, resulting in zero network lag during icon rendering.

07
Edge-to-Edge SafeArea
Hardware-accelerated window insets adaptation that prevents layout jumps when the Android keyboard or system navigation appears.

08
Strict TypeScript 6.0
Compile-time prop and state validation prevents runtime undefined errors and guarantees predictable component interfaces.

10 — NATIVE BUILD PIPELINE & OFFLINE APK

From JavaScript to standalone Android APK.

Overcoming Metro dev-server dependencies for reliable offline execution on physical hardware.

STAGE 01

Offline Bundle Generation

Pre-compiled JavaScript bundle and 19 vector drawable assets compiled into the native Android assets directory:

- assets/index.android.bundle
- res/drawable-* (19 assets)

Eliminates on-the-fly JavaScript bundling and network requests.

STAGE 02

useDevSupport = false

Configured MainApplication.kt with useDevSupport = false in getDefaultReactHost:

- Prevents Android from seeking Metro on port 8081
- Eliminates 'Unable to load script' red screens
- Forces Android to load embedded assets directly.

STAGE 03

Physical Hardware Verification

Verified on connected physical device (Vivo 1919):

- Terminated all background Metro servers
- Removed all adb reverse network tunnels
- Cold boot via am force-stop
- 0 fatal crashes, instant native render.

● Result: The user can disconnect the USB cable and open BrewBox from their app drawer anytime — 100% offline.

Rigorous verification across the React Native stack.

FLOWS TESTED ON PHYSICAL VIVO 1919 (ANDROID 12)

- ✓ Bottom Tab Navigation & Stack Transitions
- ✓ Controlled Search & Multi-Criteria Filtering
- ✓ Sensory RoastMeter 5-Stage Visual Scale
- ✓ Multi-Dimensional TasteProfileBar Meters
- ✓ Interactive 6-Stage Grind Selector
- ✓ Barista Extraction Dial-In Brew Guides
- ✓ Palate Matcher 3-Step Interactive Quiz
- ✓ Dynamic Free Shipping Progress Meter (₹999)
- ✓ Promo Code Engine (ROASTMASTER15, FIRSTBREW)
- ✓ 5-Stage Live Roastery Batch Order Pipeline

TYPESCRIPT & CODE QUALITY

```
npx tsc --noEmit && npm run lint
```

- 0 errors · 0 warnings · 100% clean strict compilation

TEST SUITE & ANDROID BUILD

Jest: 2 suites, 7/7 tests passed (cartStore & App)

- Clean Gradle assembleDebug SUCCESS (10s build time)

VERSION CONTROL & REPOSITORY

`github.com/Varadha9/BrewBox`

Commit 3d9081f · Verified clean tree · Full documentation

Cross-platform mastery through BrewBox.

ACADEMIC & PRACTICAL LEARNING

- **Cross-Platform Efficiency:** Achieved 100% logic and component reuse across views with zero platform-specific divergence.
- **Component Modularity:** Designed handcrafted artisanal UI widgets without relying on bloated generic third-party UI libraries.
- **State Management Mastery:** Implemented atomic Zustand stores for 60fps rendering without Context API re-render penalties.
- **Native Packaging:** Overcame Metro dev-server constraints by mastering Android APK bundling and native Host overrides.

Thank you — questions welcome.

FUTURE SCOPE & EXTENSIONS

01 Supabase Cloud Backend
PostgreSQL database integration for user authentication, live catalog synchronization, and cloud order persistence.

02 Razorpay Native Gateway
Production payment gateway SDK integration supporting instant UPI Intent, Cards, and Net Banking.

03 Cross-Platform iOS & iPadOS
Native iOS compilation and testing with Apple Pay and iPad split-view optimization.

04 Push Notifications via FCM
Firebase Cloud Messaging alerts notifying users of newly dropped seasonal micro-lots and roasting days.

05 Smart Bluetooth Scale Sync
Bluetooth Low Energy (BLE) integration with smart scales and kettles for real-time extraction flow-rate guidance.